

Full Stack Machine Coding Assignment

Student Result Management System

Objective

Build a full-stack Student Result Management System using React for the frontend and Node.js/Express for the backend. Students should implement CRUD operations, form validation, API integration, searching, and a statistics dashboard. The focus is on clean code, component-based architecture, REST APIs, and responsive UI.

Duration

2 Hours

Task 1: Backend Setup

Create an Express server. Create REST APIs for managing students. You may use MongoDB or an in-memory array. Implement GET /students, POST /students, PUT /students/:id, DELETE /students/:id, and GET /students/search?name=. Return proper HTTP status codes and JSON responses.

Task 2: Data Model

Each student must contain: id, name, email, course, marks. The backend should calculate pass/fail based on marks ≥ 40 or the frontend may derive it.

Task 3: Validation

Validate all requests. Name and course cannot be empty. Email must be valid and unique (if possible). Marks must be a number between 0 and 100. Return descriptive validation messages.

Task 4: Frontend Dashboard

Build a responsive dashboard that fetches students from the backend and displays them in a table. Columns: Name, Email, Course, Marks, Result, Actions.

Task 5: Add Student

Create a controlled form with Name, Email, Course and Marks. Validate before submitting. On success, refresh the table without reloading the page.

Task 6: Edit Student

Clicking Edit should populate the form with existing data. After updating, call the PUT API and refresh the list.

Task 7: Delete Student

Clicking Delete should show a confirmation dialog. On confirmation call the DELETE API and remove the record from the UI.

Task 8: Search

Provide a search bar to search students by name. Do not allow empty searches. Use the backend search endpoint.

Task 9: Statistics

Display cards showing Total Students, Passed Students, Failed Students and Average Marks.
Update automatically after CRUD operations.

Bonus

Sorting, course filter, pagination, toast notifications, loading spinner, dark mode, responsive mobile layout.

Submission

Submit frontend and backend folders with a README explaining how to run the project.

Marking

Backend CRUD-30, Validation-10, API Integration-15, Add/Edit/Delete-35, Search-5, Dashboard-5.